ENCORE

Building a Car Insurance Onboarding Flow

Build a digital experience for car insurance onboarding — comprehensive or mandatory — including a supporting API and cloud deployment.

Part A
Build and deploy a small API service that wraps our vehicle-info endpoint.

Part B
Build a working conversational flow on the Insait platform that uses your API.

Presentation: in a personal meeting — prepare a presentation and demo of your solution.

Part A — Technical specification

Endpoint

```
https://insurance-webhook-945894769129.us-central1.run.app/docs
```

Request

```
{ "license_plate": "12345678" }
```

Success response

```
{
  "success": true,
  "data": {
    "license_plate": "12345678",
    "manufacturer": "Toyota",
    "model": "Corolla",
    "year": 2020,
    "color": "White"
  }
}
```

curl example

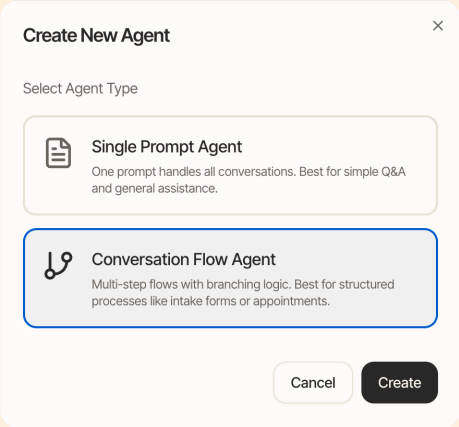
```
curl -X POST \
  https://insurance-webhook-945894769129.us-central1.run.app/vehicle-info \
  -H "Content-Type: application/json" \
  -d '{"license_plate": "12345678"}'

{"success":true,"data":{"license_plate":"12345678","manufacturer":"Toyota",
"model":"Corolla","year":2020,"color":"White"}}
```

Part B — Building the flow on the Insait platform

Build a working flow on the Insait platform for car insurance onboarding. Log in at platform.gainencore.ai and register.

Use a Conversation Flow Agent

When creating the agent, choose **Conversation Flow Agent** ("Multi-step flows with branching logic"). You'll build the onboarding as a flow graph — the user experiences a natural conversation, while the graph guarantees every required step happens.



Two approaches — strict and agentic

The platform supports two complementary ways of moving a conversation forward and capturing data:

Strict (deterministic) — you define exactly what to collect and exactly how to branch. Predictable, testable, and guaranteed: ideal for compliance-critical steps, API inputs, and validation. The trade-off is a more scripted feel.

Agentic (conversational) — built with a **conversation node** plus tools such as **save field** and **LLM exits**: you give the agent a goal and let the LLM run a natural dialogue, saving details as they come up and exiting when a described condition is met (e.g. *"the customer confirmed the vehicle details"*). Handles users who volunteer everything at once, ask side questions, or answer out of order. The trade-off is less determinism.

In node terms, briefly: **collect nodes** with typed, validated fields and **expression edges** are the strict toolkit; **conversation nodes** with saved fields and **LLM-evaluated exit conditions** are the agentic one; **API nodes** call your service with success/error routing.

Real production flows mix both — e.g. a strict opening, an agentic middle, a strict finish around the API call. How you split your flow between them is **your call** — there is no single correct answer. Be ready to explain the reasoning behind each choice.

Flow steps

1. **Opening** — welcome message; select insurance type (Comprehensive / Mandatory).
2. **Vehicle details** — collect license plate number; **call the API built in Part A**; display vehicle details for confirmation; handle errors (vehicle not found).
3. **Customer details** — full name, phone number (with validation), email (with validation).
4. **Additional coverage (Comprehensive only)** — windshield, extended third-party, replacement vehicle; multi-select.
5. **Summary & confirmation** — display all details; final confirmation.

Technical requirements

- Agent created as a **Conversation Flow Agent** (flow graph — not Single Prompt).
- API integration — success **and** error paths handled.

- Conditions for dynamic routing: deterministic edges where the branch is business logic; LLM-evaluated exits where the judgment is conversational.
- Data stored in variables, from both collect nodes and agentic saved fields.
- Input validation.
- Error handling: API not responding, validation failed, vehicle not found.

Submission

- Link to the flow, workspace name and agent name.
- A **~3-minute video recording** of you explaining the flow and running a happy path end to end.

Build it yourself

You will be asked technical questions on the flow in the interview — why each node is the type it is, how the API integration works, what happens when the user goes off-script or the API fails. Make sure you build the flow yourself and know every part of it.

Good luck!